Importing and profiling a pre-trained machine learning model on PSOC™ Edge training manual

About this document

Scope and purpose

This document is the training manual for importing and profiling a pre-trained machine learning model on PSOC™ Edge. This manual provides practical guidance for efficient deployment on PSOC™ Edge devices. It is intended for embedded ML, firmware, and application engineers who need to assess model feasibility on constrained hardware and optimise for latency, memory, and energy.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Pre-requisites **3**
3 Required development tools **4**
4 Exploring machine learning capabilities of PSOC™ Edge **5**
4.1 Objective 5
4.2 Description 5
4.3 Project creation 6
4.4 DEEPCRAFT™ Model Converter with MTB ML 3.0 10
4.5 Lab 1: Model source generation and validation 12
4.5.1 Objective 12
4.5.2 Description 13
4.5.2.1 DEEPCRAFT™ Model Converter and supported configurations 13
4.5.2.2 Model optimisation techniques 14
4.5.3 Using DEEPCRAFT™ Model Converter to import a model and generate source 16
4.5.4 Model metrics 21
4.5.4.1 Inference cycle count 21
4.5.4.2 Memory usage 21
4.5.4.3 Accuracy 21
4.6 Lab 2: Performance across PSOC™ Edge cores 22
4.6.1 Objective 22
4.6.2 Description 22
4.6.2.1 Vela compiler 22
4.6.3 Configure the project to use CM33 only 23
4.6.4 Configuring project to use CM55 and U55 NPU 24
4.7 Lab 3: Performance improvement with the model on internal memory 26
4.7.1 Objective 26
4.7.2 Description 26
4.7.2.1 PSOC™ Edge memories 26
4.7.3 Changing the memory model 27
5 Takeaways **29**
6 Conclusion **30**
7 Appendix A: KIT_PSE84_EVAL details **31**
Revision history **32**
Disclaimer **33**

1 Introduction

This manual provides instructions for creating, configuring, and building the PSOC™ Edge MCU: Machine Learning DEEPCRAFT™ Profiler code example to evaluate metrics and understand the machine learning capabilities of PSOC™ Edge.

2 Pre-requisites

Install the software and obtain the hardware listed in the Required development tools section.

This class will not cover basic concepts of ModusToolbox™ and PSOC™ Edge:

- For an introduction to PSOC™, including getting started with ModusToolbox™, go to:
  - https://www.infineon.com/product-information/psocdeveloper
- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, go to:
  - PSOC™ Edge E84 Training Collection

3 Required development tools

- ModusToolbox™ software v3.7 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2025.8.0 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- ModusToolbox™ Programming Tools v1.7.0 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- DEEPCRAFT™ Model Converter v5.8.4154 or later
  - Recommended installation via ModusToolbox™ Setup tool
- ModusToolbox™ Machine Learning Pack v3.0.1
  - Recommended installation via ModusToolbox™ Setup tool
- Board support package (BSP)
  - KIT_PSE84_EVAL_EPC2: v1.0.0 or later (available with ModusToolbox™ v3.7)
- Serial terminal emulator
  - Eclipse IDE for ModusToolbox™ includes a serial terminal that includes Tera Term, PuTTY, and so on.
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure device boots from external memory:
    - BOOT SW/SW6 in 'ON/HIGH' position
  - Disable alternate serial interface configuration:
    - Tristate/Not-Connected (NC) (J20/J21) position
  - For more details, see Appendix A: KIT_PSE84_EVAL details

Note: Other tool versions and hardware can be used with minimal or no modifications to the training manual.

4 Exploring machine learning capabilities of PSOC™ Edge

4.1 Objective

The objective of this exercise is to learn about the machine learning capabilities of PSOC™ Edge with the DEEPCRAFT™ Model Converter using the PSOC™ Edge MCU: Machine Learning DEEPCRAFT™ Profiler code example.

4.2 Description

This deep dive training aims to provide an in-depth understanding of the machine learning capabilities of the PSOC™ Edge microcontroller, leveraging the comprehensive ModusToolbox™ (MTB) and DEEPCRAFT™ ecosystem. Designed for edge AI applications, the PSOC™ Edge MCU combines rich software and hardware support, making it an ideal choice for developers looking to implement sophisticated ML solutions at the edge.

Figure 1 depicts the high-level flow of ML development and deployment using MTB ML and DEEPCRAFT™ Model Converter flow.

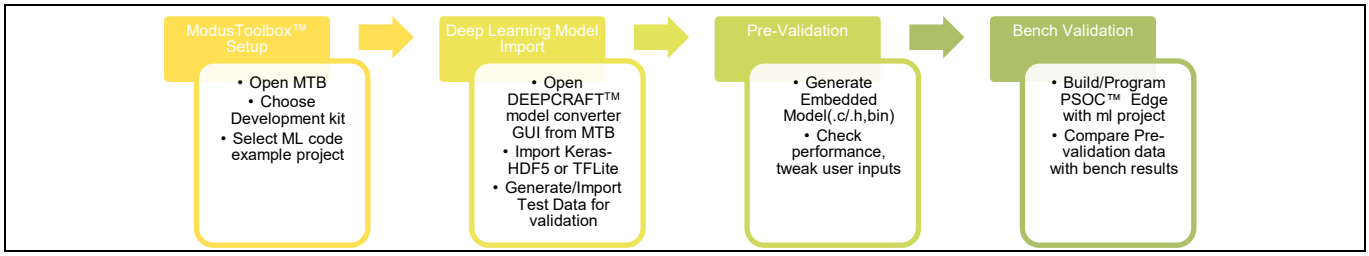

Figure 1 ModusToolbox™ machine learning (ML) and DEEPCRAFT™ dodel converter deployment flow

During this training, participants will utilise the PSOC™ Edge MCU: Machine Learning DEEPCRAFT™ Profiler code example to explore and demonstrate the practical ML functionalities of the PSOC™ Edge MCU, following the development flow depicted in Figure 1. This hands-on experience will be complemented by detailed instruction on how to use the DEEPCRAFT™ Model Converter, a graphical user interface (GUI) tool that simplifies the configuration of both machine learning software and hardware assets.

For more information on this tool, visit DEEPCRAFT™ Model Converter.

Participants will learn how to generate optimised embedded ML models, validate these models in both desktop and target environments, and compare performance metrics across different configurations. The training will also cover the integration of ML models with the PSOC™ Edge MCU's hardware accelerators, providing a clear understanding of how to maximise performance and efficiency.

By the end of this session, participants will have gained a solid foundation in using the PSOC™ Edge MCU for edge AI applications. They will be proficient in leveraging the MTB and DEEPCRAFT™ ML ecosystem to validate and optimise machine learning models, fully exploiting the capabilities of the PSOC™ Edge MCU for their specific use cases.

4.3 Project creation

1. Before creating the first project in this training series, we need to create our workspace in ModusToolbox™. Open **ModusToolbox™** from the **Windows Start** menu

   Note: If you have not installed ModusToolbox™, see the Required development tools section.

2. **Next**, choose a **workspace directory** for your project. After selecting a suitable directory, click the Launch button

   Note: It is recommended to keep the workspace path short. Windows has a 260-character path length limit, and a long workspace path may cause build issues when creating some applications.

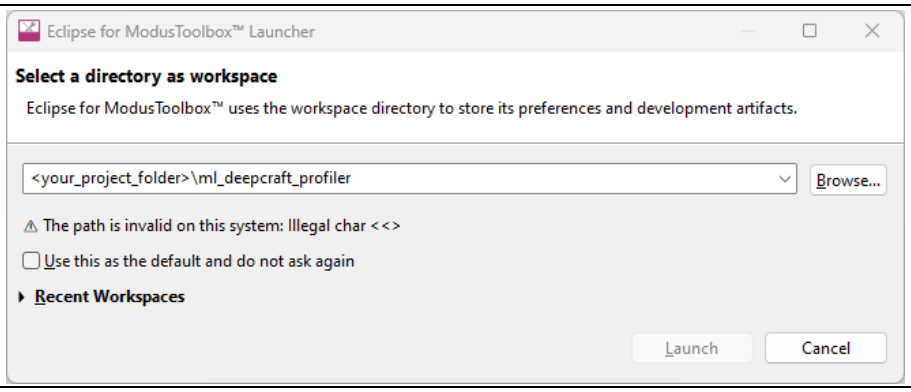

Figure 2 ModusToolbox™ - workspace selection

3. Now select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™ Application**

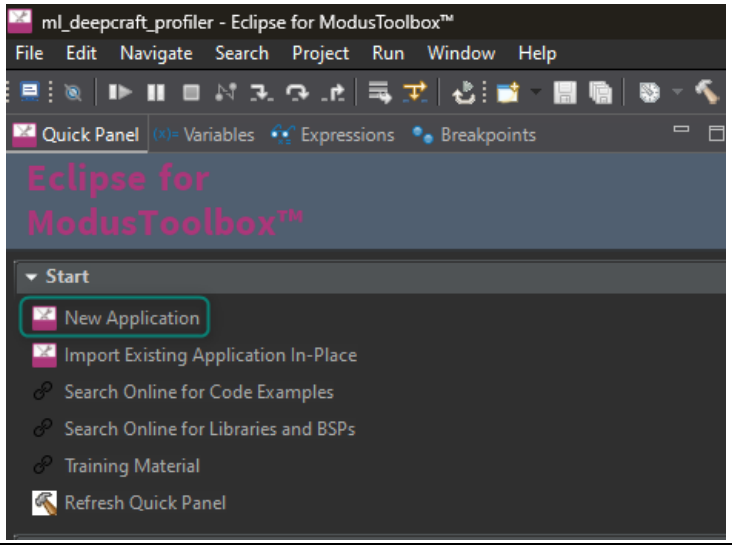

Figure 3 ModusToolbox™ - start application

4. Clicking New Application opens the **Project Creator Tool** and prompts you to choose a **board support package (BSP)**

5. Once **Project Creator** launches, select the **KIT_PSE84_EVAL_EPC2** under the **PSOC™ Edge BSPs** and click **Next**

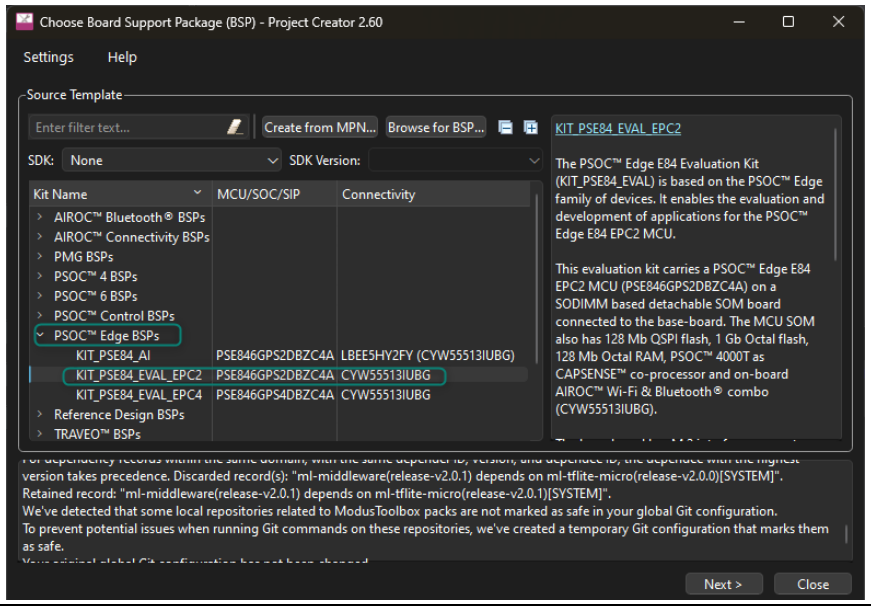

Figure 4 ModusToolbox™ - kit selection

A new window opens to select an application. Select the **PSOC™ Edge Machine Learning DEEPCRAFT™ Profiler** under the Machine Learning section. Click the **checkbox** near the application, optionally rename the project, and then click **Create.**

Note: Windows has a 260-character path length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

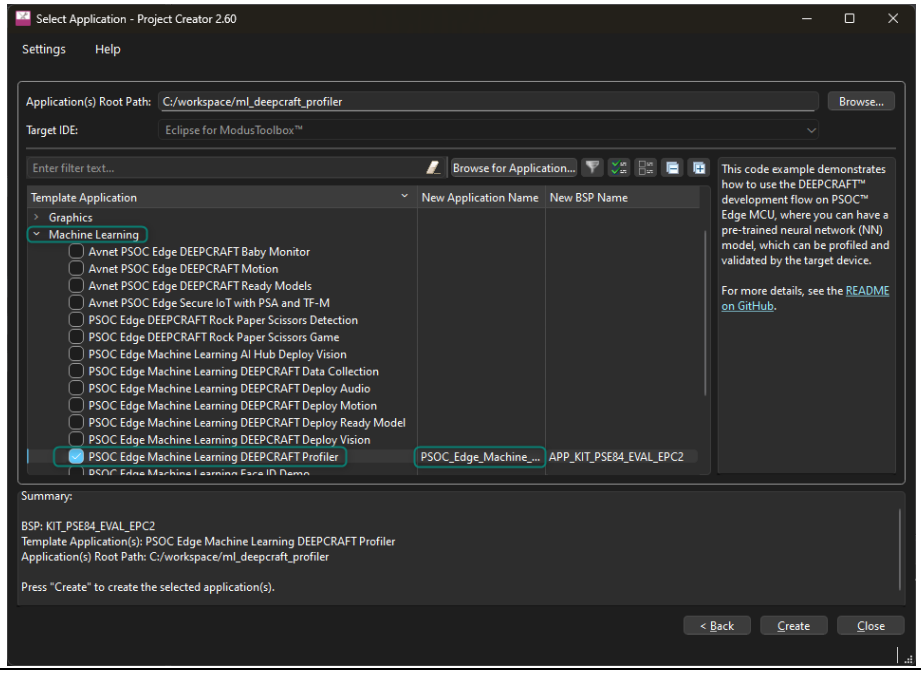

Figure 5 ModusToolbox™ - project creation

6. A new project is created, and all the files are displayed in the Project Explorer window

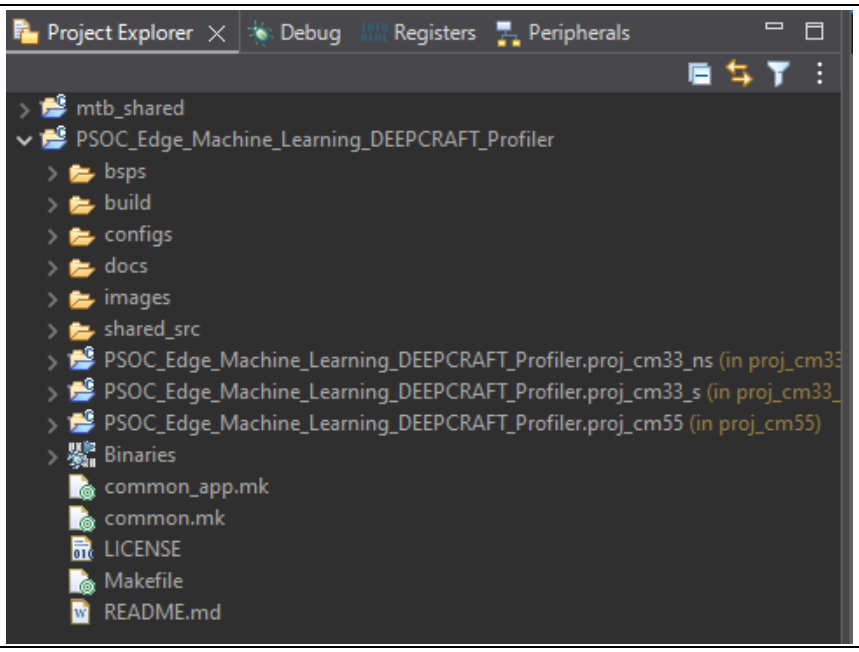

Figure 6 ModusToolbox™ - ML DEEPCRAFT™ profiler project

7. The ML DEEPCRAFT™ profiler code example supports profiling models both on CM33 or CM55, one at a time. The user can select between one of these CPU cores by setting the **ML_DEEPCRAFT_CPU** macro in **common.mk**. For the next section of the lab, set the **ML_DEEPCRAFT_CPU** to **cm33**

Code listing 1 Select CM33 CPU in common.mk

```
#Choose which CPU to run the ML profiler
#cm33
# cm55
ML_DEEPCRAFT_CPU=cm33
```

8. **[INFO]** The non-secure CM33 project (proj_cm33_ns) already has pretrained models and generated embedded models. The details of the directories containing model-related information are provided in Table 1

Table 1 ML folder structure in project

Directory name	Description
deepcraft	Contains embedded models generated using DEEPCRAFT™ Model Converter.
Deepcraft/eval_data	Contains regression data in .c/.h format created from the test data provided in the sample_data directory.
Pre-trained_models	Contains pre-trained user models in .h5 or. tflite format.
sample_data	Contains calibration data needed for model quantisation. This directory also contains test data in a supported format.

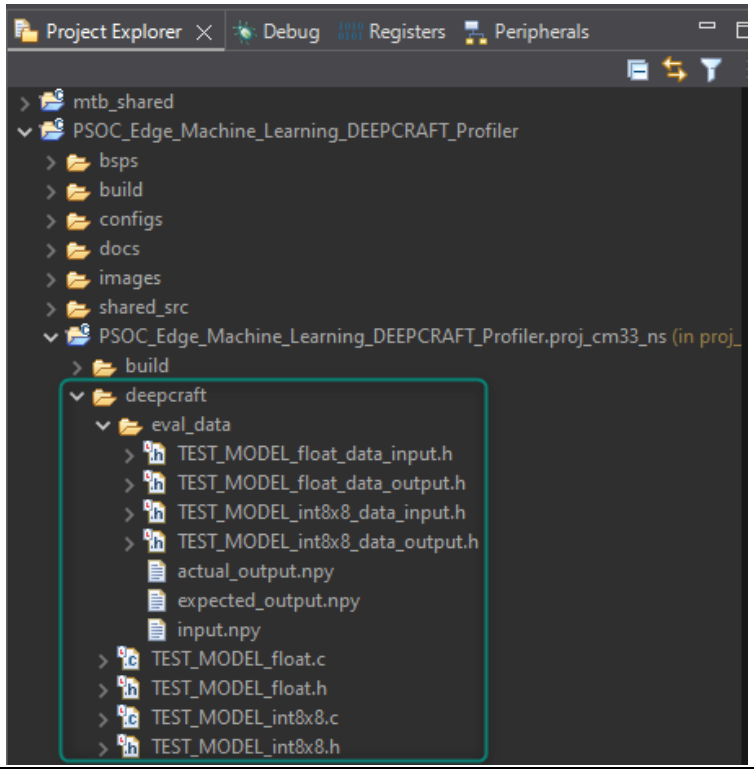

Figure 7 ML folder structure in the project

9. **[INFO]** Parameters set in the **proj_cm33_ns/Makefile** control which quantisation and model are profiled, and if the NPU (NNLite)is used. For example, the code snippet below contains configuration to run an int8x8 model:

Code listing 2 Default configuration in proj_cm33_ns/Makefile

```
# Neural Network Configuration. Options include
#
# float    -- floating point for the input data and weights
# int8x8   -- 8-bit fixed-point for the input data and weights
NN_TYPE=int8x8

# Model Name to be loaded to the firmware
NN_MODEL_NAME=TEST_MODEL

# Folder name containing the model and regression data
NN_MODEL_FOLDER=deepcraft

# Choose to enable NNLITE NPU - yes or no
NN_NPU_ENABLE=yes
```

10. Build and flash the code example and observe the output in a serial terminal ModusToolbox™ machine learning (MTB ML) DEEPCRAFT™ profiler code example prints all the metrics for the model, like model size, buffer size, average and peak CPU cycles, and then a PASS or FAIL status with the accuracy percentage. A model PASS requires an accuracy of 98% or higher

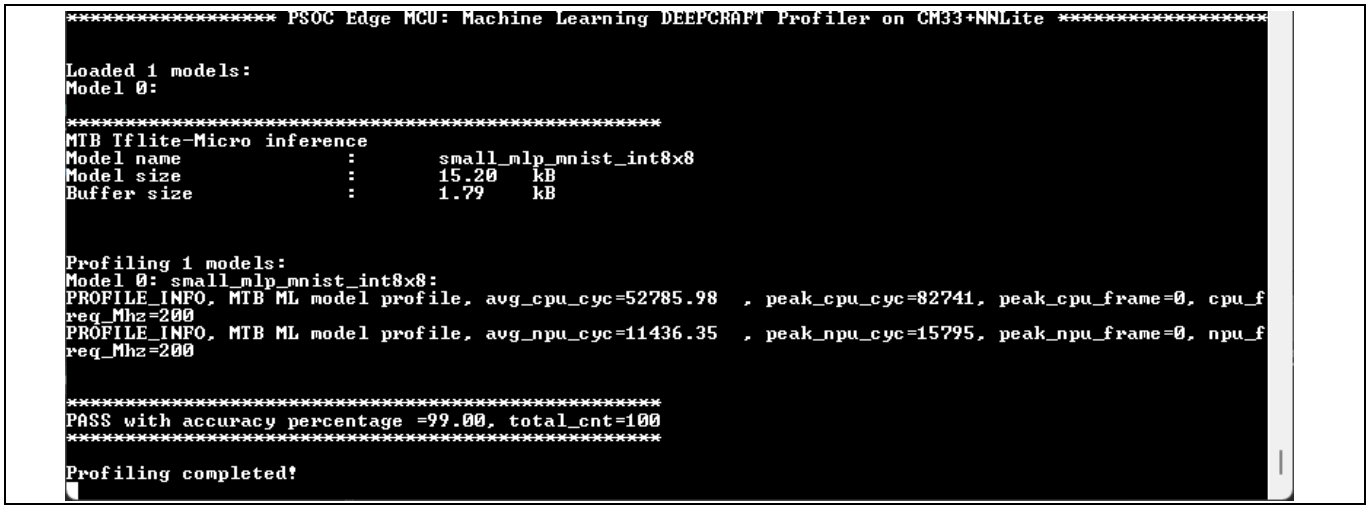

Figure 8 DEEPCRAFT™ Profiler code example output

4.4 DEEPCRAFT™ Model Converter with MTB ML 3.0

ModusToolbox™ machine learning (MTB ML) 3.0 solution is a set of tools, libraries, and middleware that will help you build, evaluate, and benchmark pre-trained ML models.

The DEEPCRAFT™ Model Converter generates model C code that can run on PSOC™ Edge. The generated code uses the assets deployed with the MTB ML 3.0 pack, including the ml-middleware and TensorFlow lite library. The Model Converter is going to replace the MTBML configurator which is currently deployed with the MTB ML 3.0 pack. There are still some limitations to the Model converter such as not supporting int16x8 quantization, interpreter-less mode, desktop validation, and streaming data to device. The Model Converter does add support to import PyTorch (pt2) models.

Table 2 shows the evolution of this tool from its inception to MTB ML 3.0.

Table 2 MTB ML features

Component	Feature	1.x	2.x	3.0
Hardware platform support	PSOC™ 6: Cortex®-M4	✓	✓	✓
	PSOC™ Edge MCU: Cortex®-M33 + NNLite	×	×	✓
	PSOC™ Edge MCU: Cortex®-M55 + Ethos-U55	×	×	✓
Model import support	Keras H5 models (.h5)	✓	✓	✓
	TFLM models (.tflite)	×	✓	✓
Inference engine with Validation	IFX: PSOC™ 6: Cortex®-M4	✓	✓	×
	TFLM: PSOC™ 6: Cortex®-M4	×	✓	✓
	TFLM: PSOC™ Edge MCU: Cortex®-M33 + NNLite	×	×	✓
	TFLM: PSOC™ Edge MCU: Cortex®-M55 + Ethos-U55	×	×	✓
Optimization	IFX: SRAM optimisation	✓	✓	×
	IFX: flat and 16/8-bit quantisation support	✓	✓	×

Component	Feature	1.x	2.x	3.0
	TFLM: Interpreter-less mode	×	✓	✓
	TFLM: float and 8-bit weights support	×	✓	✓
	TFLM: 4, 5, 6-bit for weights and 8-bit for activations	×	✓	×
	TFLM: Sparsity support	×	✓	✓
Note: The Infineon inference engine is deprecated in MTB ML 3.0				

The ModusToolbox™ ML solution consists of four major components:

- **ML Configurator (Replaced by DEEPCRAFT Model Converter)** is a GUI tool with options to load pre-trained models and test data. It also provides graphical plots and accuracy data when validating the optimised models against test data. The ML configurator relies on ml-coretools to do all the manipulations on the model and data. Based on the user-selected configuration, ML configurator generates a JSON file, which will be used by the ml-coretools. The JSON file is generated when the ML Configurator settings are saved
- **ml-coretools** is a collection of Python scripts provided as an executable. The ModusToolbox™ ml-coretools are the backend utilities that the ML Configurator interacts with to import, convert, and deploy supported embedded ML models. ml-coretools also contains a cross-domain validation tool that validates the performance of the converted ML models with respect to the original reference model and determines if the conversion process meets accuracy criteria
- **ML middleware library** includes middleware helper functions and inference engine libraries to work with an NN model and input data
- **ML code examples:** a collection of applications on how to use the ML libraries

4.5 Lab 1: Model source generation and validation

4.5.1 Objective

In this exercise, we will demonstrate how to get estimated performance from the DEEPCRAFT™ Model Converter and on a target device, comparing the accuracy of each method. The RESNET model deployed in the code example will be used for this exercise.

The user will validate performance in the Model Converter, checking the impact of quantisation.

At the end of the lab, users will observe:

- Memory and accuracy for the float and int8x8 models when generating source in the Model Converter
- Model performance when running validation data through the model on the device

Figure 9 shows the primary view of the DEEPCRAFT™ Model Converter. In the next sections, we will have an opportunity to modify and discuss these settings.

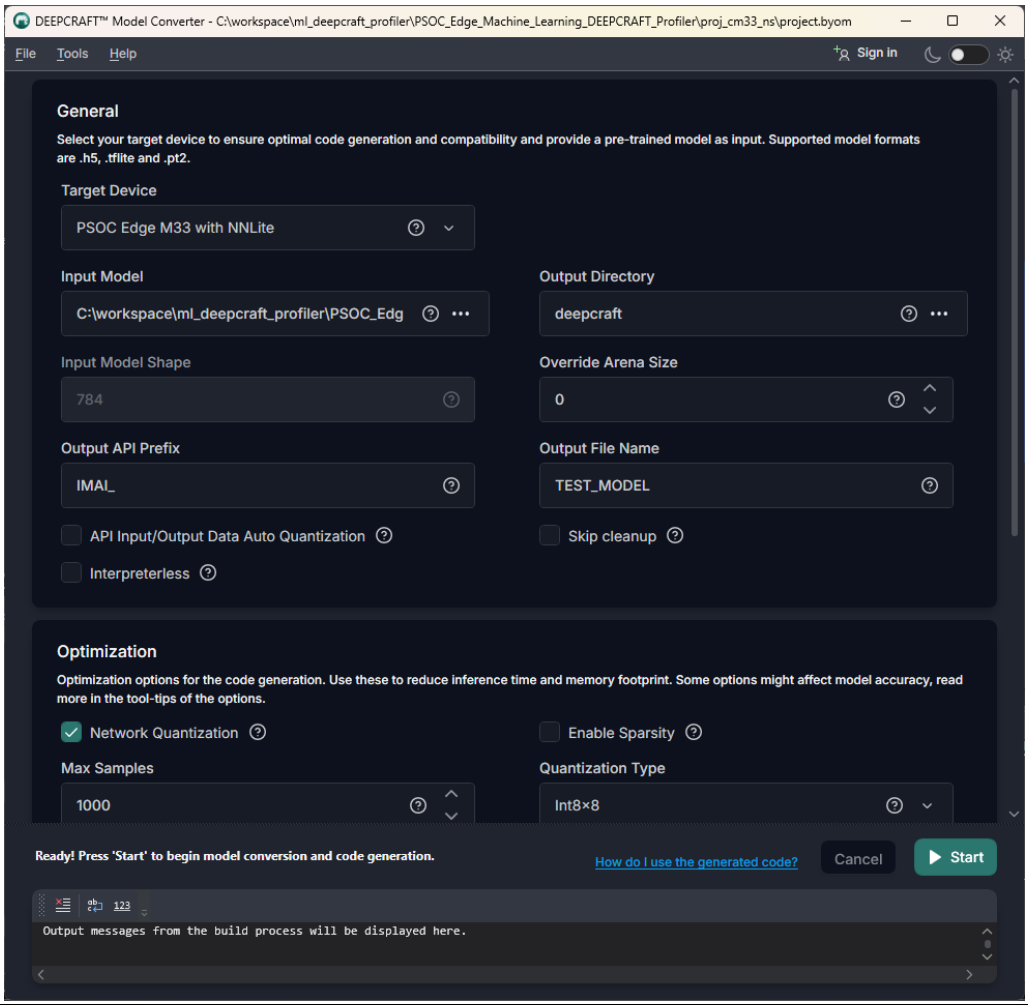

Figure 9 DEEPCRAFT™ model converter

4.5.2 Description

4.5.2.1 DEEPCRAFT™ Model Converter and supported configurations

The DEEPCRAFT™ Model Converter is used in ML applications for adapting a pretrained learning model to an Infineon target platform. The tool accepts a pretrained ML model and generates an embedded model (as a library) that can be used in a user application.

The Model Converter allows you to fit the pretrained model of choice to the target device with a set of optimisation parameters.

The NN models are available in .h5 or .tflite format. In the case of the .h5 model, the Model Converter will convert this model into .tflite format. During the conversion to .tflite, the model can be optimised using quantisation (Post Training Quantization) or sparsity if the user enables these options in the Model Converter. The user-selected configurations are stored in a JSON file and are used when generating source code. The Model Converter generates the following:

- Model files in a .c/.h for deployment on embedded devices
- Model input and output data (regression) in .c and .h format for deployment on embedded devices

Figure 10 shows a detailed flow of NN model profiling with the help of Model Converter and ML DEEPCRAFT™ Profiler code example.

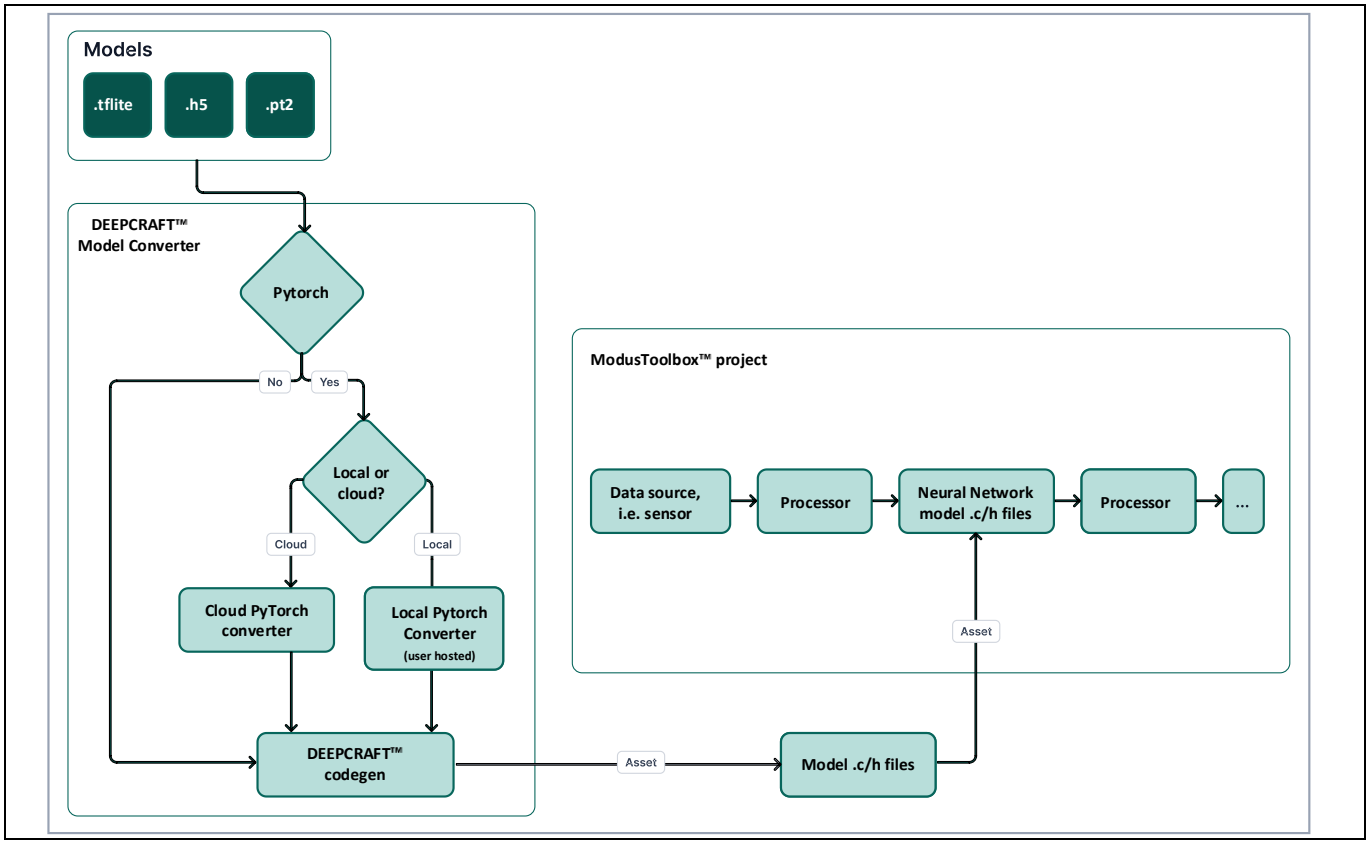

Figure 10 NN model profiling with DEEPCRAFT™ Model Converter and DEEPCRAFT™ Profiler example

Note that a Keras H5 model needs to be converted to TFLite before generating source. The model can be quantized during the conversion. Post-Training Quantization (PTQ) uses a calibration dataset to capture the minimum and maximum values of all variable tensors in your model. By capturing these ranges, it is possible to fully quantize not just the weights of the model but also the activations. The calibration data should be imported in floating point.

To evaluate converted models on the supported embedded platforms, test data created for the model evaluation needs to be converted according to the model quantization. Data is quantized to a relevant precision and then saved in flat buffer binary files and in the form of C-header files for inclusion in an embedded MTB-ML project. The Model Converter creates a set of files for the input features (X-data) and for the corresponding result (Y-data) for each supported model quantization. 'X' input data files are composed of a header with metadata and data. 'Y' output data files contain data only (no header).

For more details on each configuration, please refer to the DEEPCRAFT™ Model Converter user documentation.

4.5.2.2 Model optimisation techniques

4.5.2.2.1 Pruning and sparsity

Sparsity and pruning are techniques used in machine learning to reduce the storage space and computations required for a model.

- **Sparsity** refers to a matrix with many zeros, while pruning is a data compression technique that removes non-critical and redundant sections in neural networks. These methods can be applied to convolutional and dense layers with zero weights, resulting in a sparse matrix. Sparsity is most effective when models are pruned. Also, the sparsity feature is most effective with integer models, as it has a negligible impact on floating-point models
- **Pruning** is typically performed during model creation, with the user deciding which layers to prune and to what extent. Once a pruned model is produced, sparsity support can be enabled in tools to further reduce the model's storage size. Some NPUs can take advantage of pruned sparse matrices to minimise unnecessary operations

The TFLM inference engine can also leverage sparsity if the provided model has been pruned. However, this document does not cover guidelines for pruning during the training process, assuming the developer provides a pre-trained, pruned model.

4.5.2.2.2 Quantization

- **Quantization** is a technique used to reduce the precision of numerical data in a model, thereby decreasing the memory and computational requirements. Like pruning and sparsity, quantization aims to optimise models for efficient storage and processing on resource-constrained devices. In the quantization process, the model's weights and activations are converted from high-precision formats, such as 32-bit floating-point numbers, to lower-precision formats, like 8-bit integers. This reduction in precision can lead to a significant decrease in memory usage and accelerate computations, as lower-precision arithmetic operations are faster and require less power. Quantization can be applied to pre-trained models or during the training process itself

- **Post-Training Quantization (PTQ)** is a technique applied to a pre-trained neural network model to reduce its size and increase inference speed by converting its weights and activations from floating-point to lower-bit representations, typically 8-bit integers. This conversion is performed after the model has been fully trained, hence the name "post-training." PTQ involves minimal effort and computational resources since it does not require retraining the model. The process can include various methods such as quantization-aware calibration, where a small calibration dataset is used to gather statistics on the activations and weights to determine appropriate scaling factors. While PTQ is relatively easy to implement and can lead to significant reductions in model size and computational requirements, it may sometimes result in a noticeable drop in model accuracy, especially for complex models or tasks that are highly sensitive to precision loss. The DEEPCRAFT™ Model Converter supports post-training quantization on keras (.h5) based imported models
- **Quantization-Aware Training (QAT)** is a more sophisticated approach that integrates the quantization process into the training phase of the neural network. Unlike PTQ, QAT involves simulating the effects of quantization during the model's training, enabling the model to learn and adapt to the reduced precision environment. This is achieved by incorporating fake quantization operations, which mimic the behavior of low-bit arithmetic, during both the forward and backward passes of training. By doing so, the model parameters are optimized with quantization effects in mind, often resulting in higher accuracy compared to PTQ, especially for complex and deep networks. However, QAT requires access to the full training dataset and additional computational resources for retraining the model, which can be time-consuming and resource intensive. Despite this, QAT is preferred for applications where maintaining high accuracy is crucial while still benefiting from the advantages of quantization. The DEEPCRAFT™ Model Converter supports QAT based models in the format of a tflite (.tflite) file

Table 3 shows the resulting model based on the quantization selection in the DEEPCRAFT™ Model Converter.

Table 3 Model formats based on quantization

Supported quantization	TensorFlow Keras (.h5)	TensorFlow Lite (.tflite)	Pytorch (.pt2)
Float 32 activations + Float 32 weights	✓	✓	✓
Int8 activations + Int8 weights	✓	✓	✓
Int16 activations + Int8 weights	✓	✓	✗

4.5.3 Using DEEPCRAFT™ Model Converter to import a model and generate source

1. This lab will utilize the ML DEEPCRAFT™ Profiler code example created in the Project Creation section
2. Copy the Resnet model files from proj_cm55 to proj_cm33_ns. For uniformity, the RESNET model will be used for all labs
   a) Copy proj_cm55/**pretrained_models/resnet.h5 to proj_cm33_ns/pretrained_models**. For uniformity Resnet model is used for all Labs
   b) Copy sample data **resnet_calib_data.npz** and **resnet_test_data.npz** from **proj_cm55/sample_data to proj_cm33_ns/sample_data**

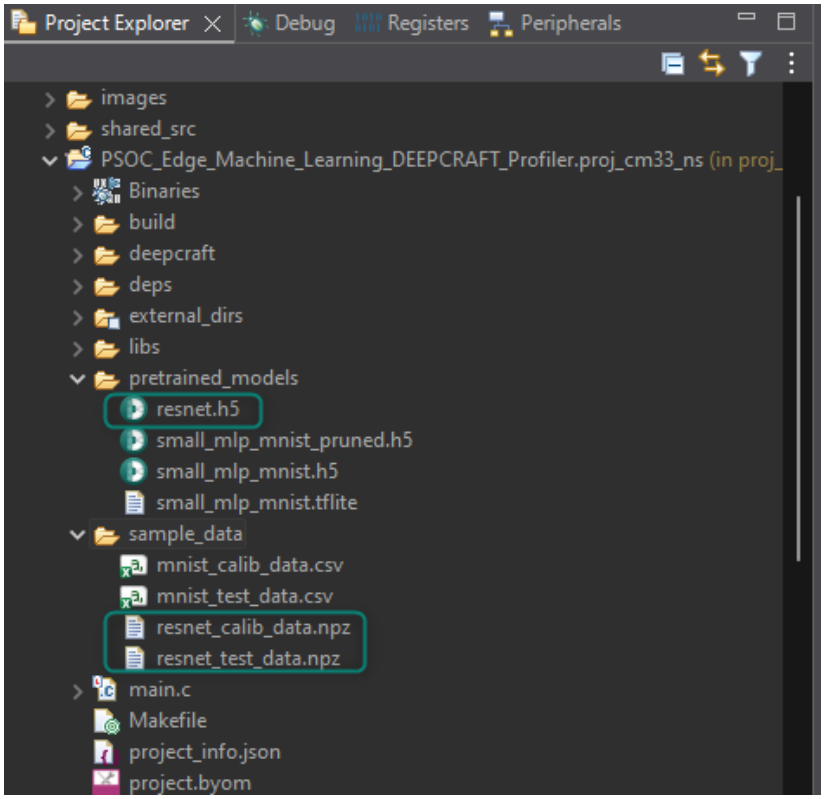

Figure 11 Resnet files copied from proj_cm55 to proj_cm33_ns

3. Open **DEEPCRAFT™ Model Converter** from quick panel of proj_cm33_ns

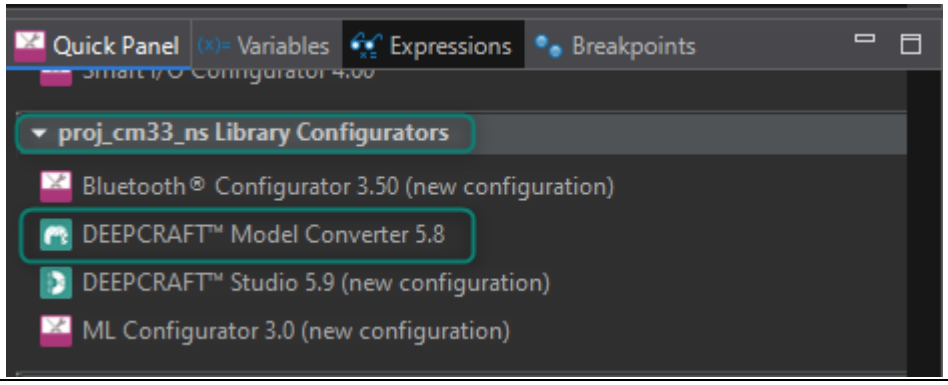

Figure 12 DEEPCRAFT™ Model Converter in the quick panel

4. Confirm that all the fields have the default values as highlighted in Figure 13. This ensures that the DEEPCRAFT™ Model Converter was opened properly for the CM33 non-secure project

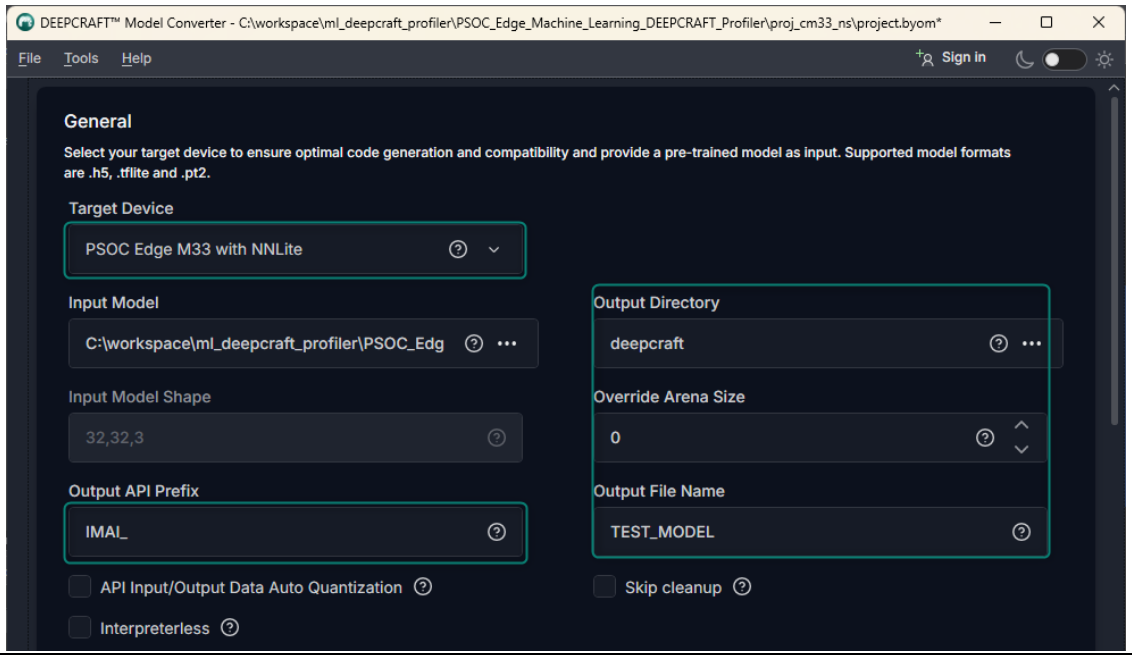

Figure 13 Default DEEPCRAFT™ Model Converter settings for proj_cm33_ns

5. In the **Input Model** field, browse and select the **resnet.h5** model copied into the **proj_cm33_ns/pretrained_model folder**

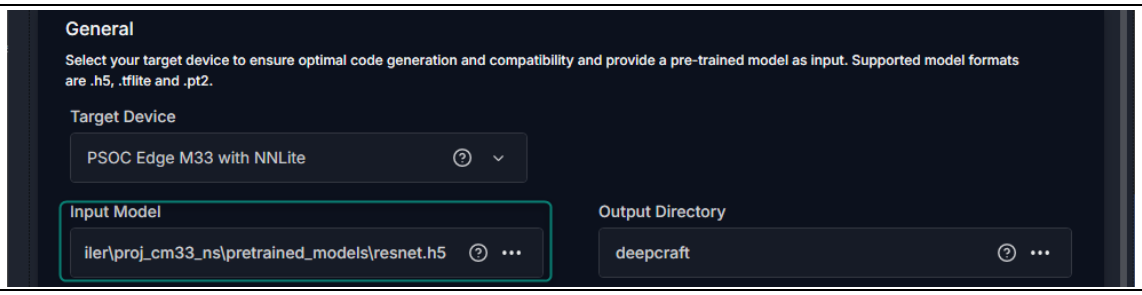

Figure 14 ML Model selection proj_cm33_ns

6. Under the **Optimization** section, deselect the **Network Quantization**, this will generate a source for a float-based model

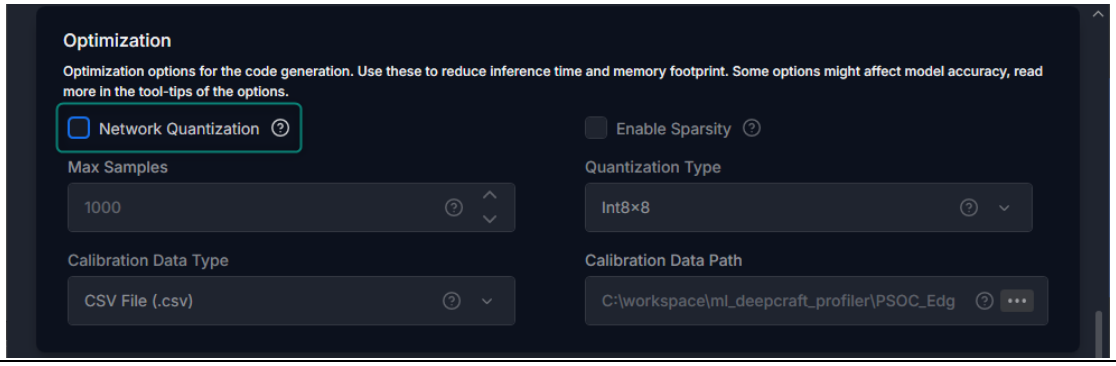

Figure 15 Disable network quantization

7. Under the **Validation** section, select **Compressed Numpy File (.npz)** option from the drop-down menu of **Validation Data Type** as shown in Figure 16. Then set the **Validation Data Path** to the **resnet_test_data.npz** file copied into sample_data directory

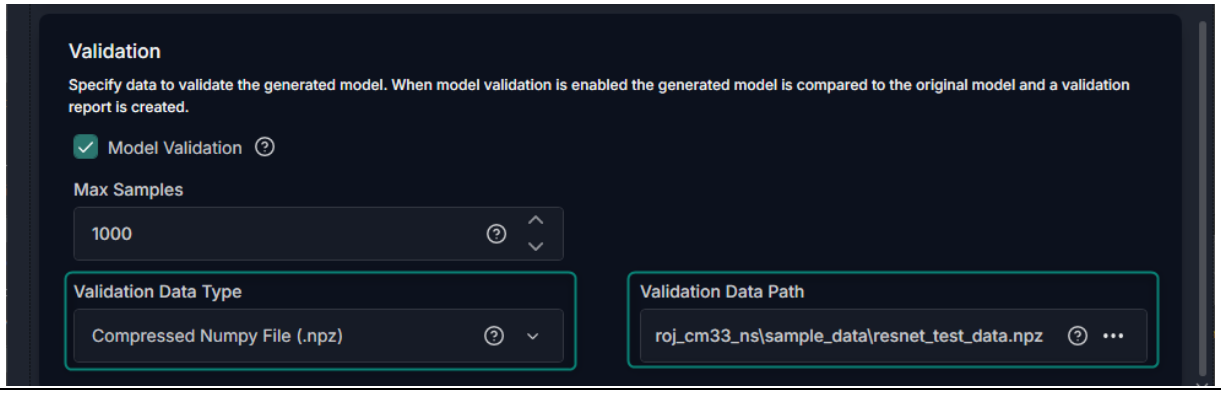

Figure 16 Select validation data for proj_cm33_ns

8. Once all the configurations are selected, generate the source by clicking the **Start** button. Wait till the source is generated. When code generation has completed, **"Code generation successful"** is printed in the terminal
9. Model metrics are printed in the terminal; scroll up to see the model memory and tensor size requirements

```
[INFO] Model metrics summary:
+---------------------+---------------------+---------------------+---------------------+---------------------+
|      Data Type      | Model Memory, bytes | Tensor Arena, bytes |      CPU Cycles     |      NPU Cycles     |
+=====================+=====================+=====================+=====================+=====================+
| float               |        316992       |        217088       |       Unknown       |       Unknown       |
+---------------------+---------------------+---------------------+---------------------+---------------------+
[INFO] Note: The NPU does not support float data format, so no operators will be executed by the NPU.
```

Figure 17 Resnet memory requirements for CM33+NNLite

10. Since model validation was enabled and configured in previous steps, the model accuracy is given at the bottom of the terminal. This shows how much accuracy was lost due to model conversion. See Model Metrics for more information

```
=== Validation Results ===

(All results are relative to the original model)

Max Absolute Error:     2.920628e-06

Mean Absolute Error:    4.745092e-08

Root Mean Square Error: 1.867663e-07

Mean Relative Error:    3.442658e-06 (0.0003%)


Classification Metrics:

(All results are relative to the original model)

Argmax Errors:          0 / 100 samples

Argmax Similarity:        100.00%
```

Figure 18 Resnet validation results for CM33+NNLite

11. Let us generate a quantized int8x8 version of the model. Under **Optimization**, enable the **Network Quantization** option. Select the **Compressed Numpy File (.npz)** option from the drop-down menu of **Calibration Data Type**, and then set the **Calibration Data Path** to the **resnet_calib_data.npz** file copied into the sample_data directory

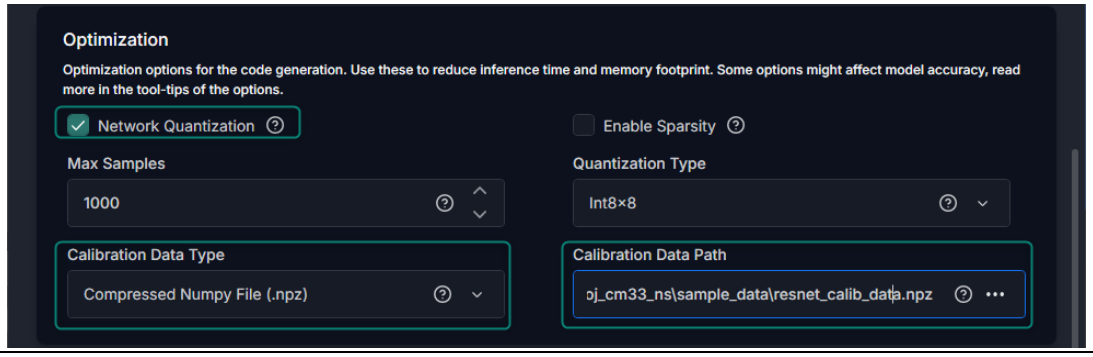

Figure 19 Enabling quantization for CM33

12. Once all the configurations are selected, generate the source by clicking the Start button. Wait till the source is generated
13. Model metrics are printed in the terminal; scroll up to see how much the model memory and tensor size were reduced due to quantization. Observe that the Model Memory was reduced by ~70% and the Tensor Arena was reduced by ~73% due to quantizing the model

```
[INFO] Model metrics summary:
+----------------------+----------------------+----------------------+----------------------+----------------------+
|      Data Type       | Model Memory, bytes  | Tensor Arena, bytes  |      CPU Cycles      |      NPU Cycles      |
+======================+======================+======================+======================+======================+
| int8x8               |        94088         |        59392         |       Unknown        |       Unknown        |
+----------------------+----------------------+----------------------+----------------------+----------------------+
```

Figure 20 Resnet memory requirements for CM33+NNLite with quantization enabled

14. When quantizing a model, the accuracy of the model can be reduced. Look at the model accuracy in the terminal to see the accuracy reduction due to model conversion and quantization. See Model Metrics for more information. Observe that there is a 1% model accuracy reduction across the 100 test samples

```
=== Validation Results ===

(All results are relative to the original model)

Max Absolute Error:     2.271660e-01

Mean Absolute Error:    5.504997e-03

Root Mean Square Error: 2.192238e-02

Mean Relative Error:    8.121618e-01 (81.2162%)


Classification Metrics:

(All results are relative to the original model)

Argmax Errors:          1 / 100 samples

Argmax Similarity:        99.00%
```

Figure 21 Resnet validation results for CM33+NNLite with quantization enabled

15. **[INFO]** Observe the generated files at **..\proj_cm33_ns\deepcraft**. Open one of the .c files and observe the NN model in array form, containing weights and biases

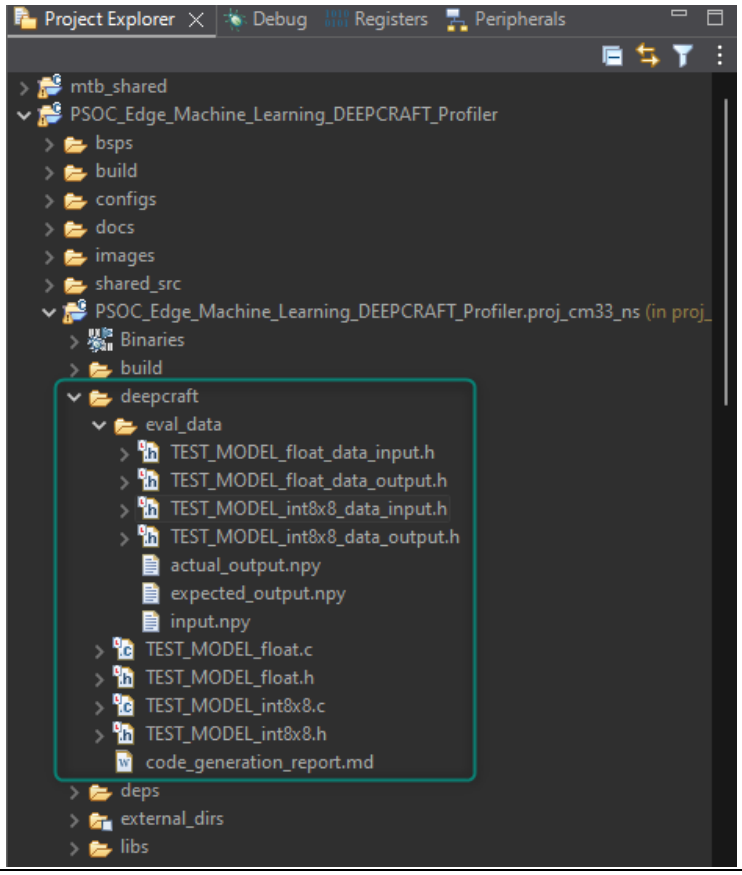

Figure 22 Generated files for proj_cm33_ns

16. Build and flash the code example
17. Open the serial terminal and set the baud rate to 115200. Observe that the application prints out the actual memory used and cycles taken to complete one inference. The data used is the same validation data that was passed into the Model Converter. Make note of the average and peak cycles for both the CPU and NPU

    Note: The model metrics given in the Model Converter are estimates and will vary slightly when running on a device.

```
****************** PSOC Edge MCU: Machine Learning DEEPCRAFT Profiler on CM33+NNLite ******************

Loaded 1 models:
Model 0:

****************************************************
MTB Tflite-Micro inference
Model name              :       resnet_int8x8
Model size              :       91.88    kB
Buffer size             :       52.02    kB


Profiling 1 models:
Model 0: resnet_int8x8:
PROFILE_INFO, MTB ML model profile, avg_cpu_cyc=204190.12 , peak_cpu_cyc=217329, peak_cpu_frame=0, cpu_freq_Mhz=200
PROFILE_INFO, MTB ML model profile, avg_npu_cyc=5140238.50, peak_npu_cyc=5143065, peak_npu_frame=0, npu_freq_Mhz=200

****************************************************
FAIL with accuracy percentage =97.00, total_cnt=100
****************************************************

Profiling completed!
```

Figure 23 Model metrics running on CM33+NNLite

Table 4 Cycle count for CM33+NNLite

Core	Average cycle count	Peak cycle count
CM33 only	–	–
CM33+NNLite	**CM33: 204,190** **NNLite: 5,140,238**	**CM33: 217,329** **NNLite: 5,143,065**
CM55+U55	–	–

4.5.4 Model metrics

4.5.4.1 Inference cycle count

The cycle count estimator allows to estimate model's inference time in CPU and/or NPU cycles as average and peak cycles.

Currently, the Model Converter supports giving cycle estimates for the U55 only. To get cycle counts for the CM33, NNLite, and CM55, inference must be done on the device. The cycle count for U55 is taken from the Vela compiler's output.

4.5.4.2 Memory usage

When using the TFLM inference engines, different types of memory blocks are required, and special attention is required to allocate them.

The memory blocks required are:

- **Model memory (bytes):** Contains the model weights
- **Tensor Arena (bytes):** A combination of scratch and persistent memory, handled automatically by the TLFM inference engine

The developers can select the memory type for both the Model and Tensor arenas individually. This will be demonstrated in the hands-on labs.

4.5.4.3 Accuracy

In classification tasks, accuracy is a fundamental metric that measures the proportion of correctly predicted instances out of the total instances in the dataset. It is calculated as the ratio of the sum of true positives and true negatives to the overall number of predictions, providing a straightforward assessment of a model's performance. However, it can sometimes be misleading, especially with imbalanced datasets, where other metrics like precision, recall, and F1-score may offer more insight.

In regression tasks, accuracy is not typically used; instead, performance is evaluated using metrics like Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2). These metrics assess how closely the model's predicted values match the actual values, focusing on the magnitude of prediction errors rather than simply counting correct predictions. Each metric has its own strengths and is chosen based on the specific requirements and nuances of the regression problem at hand.

4.6 Lab 2: Performance across PSOC™ Edge cores

4.6.1 Objective

In this lab, you will do model profiling on CM33 only and CM55 + U55 cores. By noting down the cycle count for each configuration, you will observe the difference between the CM33 only, CM33+NNLite (from Lab 1) and CM55 + U55 cycle count.

4.6.2 Description

In Lab 1, we ran the Resnet model using CM33 and NNLite NPU, and we observed the difference between enabling and disabling quantization, which is a setting available in the DEEPCRAFT™ Model Converter.

In this lab, we will dive deeper into the differences between executing on different CPUs and NPUs. We will observe the dramatic difference between enabling and disabling the NPU, and then we will observe the advantages of utilising the U55 NPU for complex models.

4.6.2.1 Vela compiler

The Vela compiler tool is used to compile a TensorFlow Lite for microcontrollers neural network (.tflite) models into an optimised version that can run on an embedded system containing an Arm® Ethos-U55 NPU. To be accelerated by the Ethos-U55 NPU, the network operators must be quantized to 8-bits (unsigned or signed). The optimised model will contain TensorFlow Lite custom operators for those parts of the model that can be accelerated by the Ethos-U55 NPU. Parts of the model that cannot be accelerated are left unchanged and will instead run on the Cortex®-M55 CPU using an appropriate kernel (such as the Arm® optimized CMSIS-NN kernels). After compilation, the optimized model can only be run on an Ethos-U NPU embedded system.

The DEEPCRAFT™ Model Converter contains a separate panel for the Vela compiler options. The **Ethos U Options** panel will be visible in the Model Converter for PSOC™ Edge M55 with the U55 target device. For the CM55+U55 core option, the Model Converter provides additional options related to the Vela compiler, which are explained in Table 5.

Table 5 Vela compiler options in model converter

Parameter	Options	Description
System configuration	SOCMEM 300 MHz QUAD XIP	Generates source assuming SOCMEM runs at 300 MHz
	SOCMEM 200 MHz QUAD XIP	Generates source assuming SOCMEM runs at 200 MHz
Memory mode	SRAM only	Default option. Weights and tensor arena are placed in System SRAM (SOCMEM)
	Shared SRAM	Weights are placed in the external memory and tensor area is placed in System SRAM (SOCMEM)
Optimize	Performance	Default option. Optimizes the model for best performance
	Size	Optimizes the model for minimum size
Tensor allocator	HillClimb	Default option. Heuristic-based approach that aims to minimize memory fragmentation and reduce peak memory usage during model execution
	Greedy	Allocates tensors based on their size and the availability of free memory

Parameter	Options	Description
	LinearAlloc	Allocates tensors one after another in a contiguous block of memory
Arena cache size (Bytes)	<Integer number>	Set the size of the arena cache memory area, in bytes.
Recursion limit	<Integer number>	Default is 1000. Set the recursion depth limit.
Max block dependency	1, 2 or 3	Default is 3. Set the maximum value that can be used for the block dependency between NPU kernel operations.

4.6.3 Configure the project to use CM33 only

1. Using the same project from Lab 1, open **proj_cm33_ns/Makefile** and change the **NN_NPU_ENABLE** to **no**, then save the file. This will configure the model to run on the CM33 only. In Lab 1, the model was run using the CM33+NNLite

Code listing 3 Disable NNLITE NPU in Makefile

```
# Choose to enable NNLITE NPU - yes or no
NN_NPU_ENABLE=no
```

2. Build and flash the code example
3. Observe the output in the serial terminal

```
****************** PSOC Edge MCU: Machine Learning DEEPCRAFT Profiler on CM33+NNLite ******************

Loaded 1 models:
Model 0:

*************************************************
MTB Tflite-Micro inference
Model name              :       resnet_int8x8
Model size              :       91.88    kB
Buffer size             :       53.89    kB


Profiling 1 models:
Model 0: resnet_int8x8:
PROFILE_INFO, MTB ML model profile, avg_cpu_cyc=42191164.00, peak_cpu_cyc=42197220, peak_cpu_frame=0, cpu_freq_Mhz=200


*************************************************
PASS with accuracy percentage =99.00, total_cnt=100
*************************************************

Profiling completed!
```

Figure 24 Profiler output using CM33

4. Create a table as in Table 6 and enter the average and peak cycle count for the M33 only row. Add the average and peak cycle count for the M33+NNLite data taken from the Lab 1

Table 6 Cycle count for CM33 and CM33+NNLite

Core	Average cycle count	Peak cycle count
CM33 only	CM33: 42,191,164	CM33: 42,197,220
CM33+NNLite	CM33: 204,190 NNLite: 5,140,238	CM33: 217,329 NNLite: 5,143,065
CM55+U55	–	–

4.6.4 Configuring project to use CM55 and U55 NPU

1. Open **common.mk** and change the **ML_DEEPCRAFT_CPU** to **cm55**

Code listing 4 Enable CM55 CPU in common.mk

```
# Choose which CPU to run the DEEPCRAFT ML profiler
# cm33
# cm55
ML_DEEPCRAFT_CPU=cm55
```

2. Open DEEPCRAFT™ Model Converter from quick panel for **proj_cm55**

*Note: Confirm that right file is opened, by checking that the Target Device is set to **PSOC™ Edge M55 with U55**.*

3. Ensure the **Ethos U Options** are configured shown in Figure 25

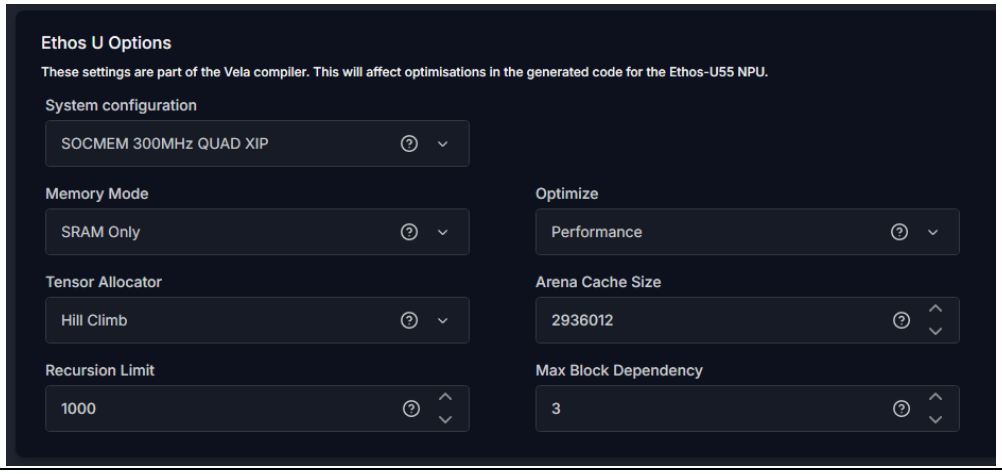

Figure 25 Vela compiler options using CM55 + U55

4. Click on **Start** button and wait till source is generated
5. Build and flash the code example

6. Observe the output on the terminal

```
****************** PSOC Edge MCU: Machine Learning DEEPCRAFT Profiler on CM55+U55 ******************

Loaded 1 models:
Model 0:

****************************************************
MTB Tflite-Micro inference
Model name              :       resnet_int8x8
Model size              :       85.50   kB
Buffer size             :       48.57   kB


Profiling 1 models:
Model 0: resnet_int8x8:
PROFILE_INFO, MTB ML model profile, avg_cpu_cyc=88344.66  , peak_cpu_cyc=120287, peak_cpu_frame=0, cpu_freq_Mhz=400
PROFILE_INFO, MTB ML model profile, avg_npu_cyc=183088.98 , peak_npu_cyc=186437, peak_npu_frame=0, npu_freq_Mhz=400


****************************************************
PASS with accuracy percentage =99.00, total_cnt=100
****************************************************

Profiling completed!
```

Figure 26 ML DEEPCRAFT™ Profiler output using CM55 + U55

7. Enter the average and peak cycle count for the M55 + U55 row. Observe the difference between the cycle count as different cores/NPU are used. It is clearly visible that performance is greatly improved with the U55 NPU use. The performance increase will vary depending on the model. To complete one inference, the Resnet model takes ~211ms using the CM33 at 200 MHz, ~26.7 ms using the CM33+NNLite at 200 MHz, and ~0.68ms using the CM55+U55 at 400 MHz

Table 7 Cycle count for CM33 and CM33+NNLite

Core	Average cycle count	Peak cycle count
CM33 only	CM33: 42,191,164	CM33: 42,197,220
CM33+NNLite	CM33: 204,190 NNLite: 5,140,238	CM33: 217,329 NNLite: 5,143,065
CM55+U55	**CM55: 88,344** **U55: 183,088**	**CM55: 120,287** **U55: 186,437**

4.7 Lab 3: Performance improvement with the model on internal memory

4.7.1 Objective

In this exercise, we will demonstrate the performance improvement using the internal memory for the model.

4.7.2 Description

4.7.2.1 PSOC™ Edge memories

The PSOC™ Edge MCU provides a diverse range of internal and external memories. Table 8 provides an overview of the various memory types available in the PSOC™ Edge MCU, their respective use cases, and the associated power domains.

Table 8 Internal and external memories on PSOC™ Edge

Memory	Size	Power domain	Description
SRAM	1 MB	PD0	SRAM memory is used for storing data, variables, and buffers required for runtime operations. The MCU can read, write, and manipulate data during program execution.
RRAM	Up to 512 KB	PD0	RRAM is typically used to store CPU instructions and data when the device power is off. It offers high-bulk program performance and supports ultra-low-power operation.
System SRAM (SoCMEM)	Up to 5 MB	PD1	System SRAM in the PD1 domain is used for storing data, variables, and buffers required for runtime operations.
ITCM	256 KB	PD1	This can be utilized for implementing essential routines like interrupt handling or real-time tasks, guaranteeing maximum responsiveness without any cache latency.
DTCM	256 KB	PD1	To achieve faster, predictable, and deterministic data access, the CPU core is directly linked to DTCM for storing critical buffers and efficiently managing context-switching data in systems with rapid context switching.
SMIF0/1	64 MB/64 MB	PD1	Extend the memory used for code or data by 1 GB per SMIF, or 64 MB per SMIF core when used in XIP mode.

Each memory location has its own associated wait states. A memory's wait state is the number of clock cycles that the CPU must wait to access data from memory. This delay occurs when the memory is unable to respond to the CPU's request within a single clock cycle. By using faster memory, the CPU can access data more quickly, resulting in improved system responsiveness and efficiency.

Less latency will result in better performance, but this is not the only consideration when allocating memory. It is also important to consider other trade-offs, such as available size and power consumption.

It is recommended to use internal memories over external ones for model storage.

Table 9 Weights and Tensor arena placement for best performance

Inferencing core	Weights	Tensor arena
M33	SRAM	SRAM
M55+U55	System SRAM (SOCMEM)	System SRAM (SOCMEM)

When inferencing on the M55, SOCMEM gives the best performance. While the DTCM and ITCM memories have no wait states, the U55 cannot access these memory locations.

4.7.3 Changing the memory model

The memory model in Lab 2 was using System SRAM (SOCMEM), so we already have the corresponding cycle count for running an inference on CM55+U55 target. We will now observe the difference running the model from other memories.

1. Open **proj_cm55/Makefile** and set the **CY_ML_MODEL_MEM** to **.constdata**. This will only place the model weights into flash, the tensor arena is still placed in SOCMEM

Code listing 5 Using external memory for model weights

```
# Enable model profiling and store the tensor and weights in socmem
ifeq (cm55, $(ML_DEEPCRAFT_CPU))
DEFINES+=IMAI_PROFILING
DEFINES+=CY_ML_MODEL_MEM=.constdata
DEFINES+=CY_ML_ARENA_MEM=.cy_socmem_data
Endif
```

2. Open the DEEPCRAFT™ Model Converter for proj_cm55 and update the **Memory mode** to **Shared SRAM**. This will generate a more optimal code when running from flash. Save and click on **Start**

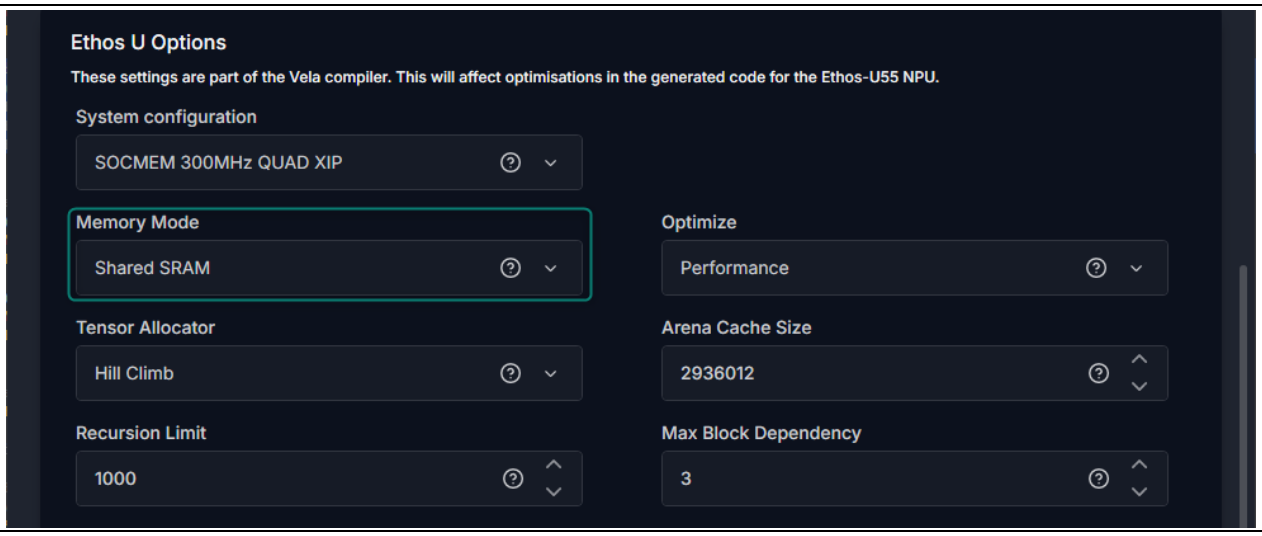

Figure 27 Changing memory mode for U55

3. Build and flash the code example

4. Observe the output on the terminal

```
****************** PSOC Edge MCU: Machine Learning DEEPCRAFT Profiler on CM55+U55 ******************

Loaded 1 models:
Model 0:

*****************************************************
MTB Tflite-Micro inference
Model name              :       resnet_int8x8
Model size              :       86.53   kB
Buffer size             :       51.19   kB


Profiling 1 models:
Model 0: resnet_int8x8:
PROFILE_INFO, MTB ML model profile, avg_cpu_cyc=92116.62   , peak_cpu_cyc=131000, peak_cpu_frame=0, cpu_freq_Mhz=400
PROFILE_INFO, MTB ML model profile, avg_npu_cyc=732748.81 , peak_npu_cyc=738464, peak_npu_frame=0, npu_freq_Mhz=400


*****************************************************
PASS with accuracy percentage =99.00, total_cnt=100
*****************************************************

Profiling completed!
```

Figure 28 ML Profiler output using Shared SRAM memory mode

5. Enter the average and Peak cycle count for the “Model on Ext. Flash” column

Note: Observe that if model weights are placed on internal memories which has less latency, then the performance is improved by approximately 3 times.

Table 10 Comparison running model on CM55+U55 from System SRAM vs. external flash

Core	Model on system SRAM Tensor arena on system SRAM		Model on external flash Tensor arena on system SRAM		X fold improvement with model in system SRAM (Average cycle count on ext. flash mem/ Average cycle count on internal mem)
	Avg. cycles	Peak cycles	Avg. cycles	Peak cycles	
CM55+U55	CM55: 88,344 U55: 183,088	CM55: 120,287 U55: 186,437	CM55: 92,116 U55: 732,748	CM55: 131,000 U55: 738,464	3.03

5 Takeaways

- The DEEPCRAFT™ Model Converter provides model metrics when generating model source. The PSOC™ Edge MCU: Machine Learning DEEPCRAFT™ Profiler allows users to deploy and test models on device without having to write code
- Leveraging the NPU available in PSOC™ Edge accelerates ML computations, enhancing performance and significantly reducing power consumption
- Placing the model in internal memory considerably improves performance and further conserves power

6 Conclusion

In this training, we explored various optimization techniques for neural network models, including pruning, sparsity, and quantization, to enhance performance, reduce power consumption, and make them suitable for deployment on edge devices.

We examined the capabilities of the Ethos-U55 NPU and the importance of using internal memory for model storage.

Understanding and applying these concepts will enable you to optimize and deploy efficient machine learning models in resource-constrained environments, ensuring real-time processing and cost-effective scalability.

7 Appendix A: KIT_PSE84_EVAL details

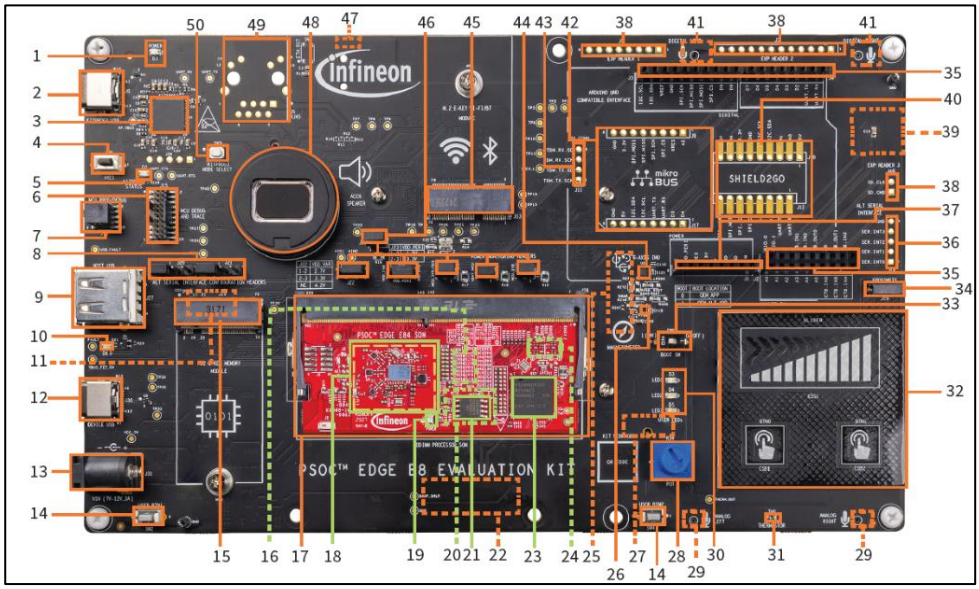

Figure 29 **PSOC™ Edge EVK details**

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C power delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128 Mbit Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor System on Module (SoM) 260-pin SODIMM connector (J28)
18 CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor System on Module (SoM) power LED (D3)
20 1 Gbit Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128 Mbit Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon's Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

*Footprint only, not populated on the board
**Component at the bottom side of the Baseboard
***Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-03-15	Initial release.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.
PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-03-15

Published by

Infineon Technologies AG

Am Campeon 1-15

85579 Neubiberg

Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.
For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.
Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.
The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.
Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:
Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.
If this document includes or references software:
The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.
If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com